Знание вне промта

Процессы на больших языковых моделях: четвёртое занятие дуги.

Что остаётся от прошлого занятия

Процесс, с которым вы ушли с прошлого занятия, умел пять вещей: отвечать обратившемуся вместо того, чтобы молчать; проверять собственный ответ перед отправкой; решать по сумме возврата правилом, записанным в коде, а не угадыванием модели; отвечать в одном устойчивом формате на каждый вызов; и, если к заявке приложена фотография, читать её, а не просто сохранять как вложение.

Вопрос, которым закончился С9, дословно: «При каком размере и изменчивости базы знаний „всё в промт“ перестаёт быть разумным?»

Учебный центр Ольги не стоит на месте: к базе каждую неделю что-то добавляется — новый регламент, обновлённый FAQ, разбор очередного спорного случая. Десять пунктов С9 — это база на момент запуска процесса, не её постоянный размер.

Ось занятия: откуда процесс берёт факты и почему считает их верными

ВЫГЛЯДИТ НАДЁЖНЫМ	ЧЕМ ОКАЗЫВАЕТСЯ
если база выросла — просто перечислим её всю в промте, как на С9	
порезать документ на части можно как угодно, главное чтобы кусок попал в хранилище	
поиск по смыслу (вектор) и так находит то же самое, что поиск по словам, только умнее	
если умеем искать текст — картинку найдём тем же способом, просто подписав её словами	

Откуда процесс берёт факты и почему считает их верными?

Четыре строки — четыре кейса занятия. Правая колонка заполняется измеренным: прогоном нашего собственного стенда на реальных данных.

Четыре кейса и один разбавитель

1. **Хранение базы знаний** — что происходит с точностью и стоимостью ответа, когда база растёт с десяти пунктов до сотен документов.
2. **Разбиение документа на части** — если база знаний больше не десять строчек, а настоящие документы, документ нужно сначала разрезать; от места разреза зависит, останется ли ответ на вопрос целым.
3. **Чем искать нужный кусок** — кусков в базе становится много, и способ поиска (по словам или по смыслу) не взаимозаменяем.
4. **Поиск по изображениям** — та же задача хранения и поиска, только для сканов и фотографий, которые учебный центр получает в заявках.

Разбавитель: система, нашедшая топически релевантный фрагмент, не обязана дать верный ответ — «найдено» и «верно» не синонимы.

Кейс 1. Хранение базы знаний

Справочник учебного центра растёт каждую неделю: новый регламент, обновлённый пункт FAQ, разбор очередного спорного случая. Десять пунктов C9 — не потолок, а стартовая точка.

Хранение базы знаний | Разбиение документа на части | Чем искать нужный кусок | Поиск по изображениям

Что Ольге нужно от растущей базы

Потребность. Ответ по-прежнему точный и дешёвый, даже когда база вырастет с десяти пунктов до сотен документов — регламентов, FAQ, переписки с прошлыми потоками.

1. Версия 1, естественное продолжение прошлого занятия: просто дописывать базу в тот же промт, без ограничения сверху. Так делали на C9, и так и предполагает делать Ольга, пока никто не проверил, что происходит на бóльшем объёме.

2. Рядом стоят ещё два реально рассматриваемых способа устроить то же самое: искать в базе по совпадению слов и искать по смыслу — через отдельное хранилище, куда база заранее превращается в числовые представления смысла.

Все три — не придуманное сравнение ради занятия, а то, что инженер в такой же ситуации реально рассматривает. Какой из них выдержит рост — вопрос следующего экрана.

Как устроить базу знаний, которая растёт

Как устроить базу знаний, которая растёт?

продолжать дописывать базу в тот же промт

искать по ключевым словам

положить в векторное хранилище и вытаскивать нужный кусок

Что показал прогон на четырёх размерах базы

РАЗМЕР БАЗЫ	ПРОМТ ЦЕЛИКОМ (ACCURACY · $/ВОПРОС)	ПОИСК — KEYWORD / VECTOR (ACCURACY · $/ВОПРОС · RECALL@1)
10	0,8 · $0,000196	0,9 / 0,9 · $0,00005 · **1,0**
50	0,6 · $0,00081	0,9 / 0,8 · $0,00005 · **1,0**
200	0,4 · $0,00328	0,7 / 0,9 · $0,00005 · **1,0**
500	**ОШИБКА — запрос отклонён**	0,8 / 0,7 · $0,00005 · **1,0**

На 500 документах запрос требует ~160 000 токенов — больше, чем принимает контекст gpt-4o-mini (128 000). OpenRouter отклоняет его с HTTP 400 ещё до того, как модель что-то прочитала.

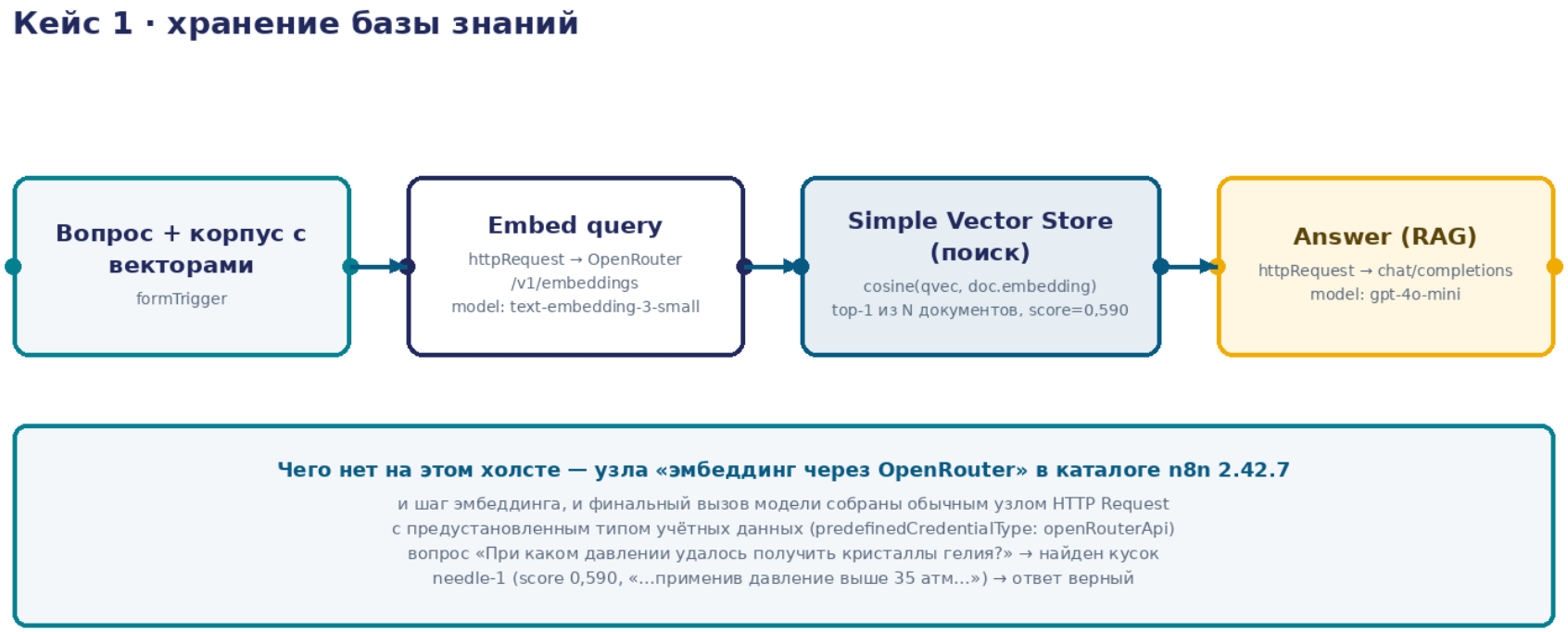
Что делает каждое из трёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Дописывать всё в промт	Весь текст базы на каждый вопрос заново уходит модели целиком	На малой базе (10 документов) просто и работает: $0,0002 за вопрос, accuracy 0,8	Accuracy падает монотонно с ростом базы — 0,8 → 0,6 → 0,4 — при росте стоимости в 16,7 раза; на 500 документах не работает вовсе — запрос превышает контекст модели
Искать по ключевым словам	Пересечение слов запроса и документа, без модели	Дёшево и быстро независимо от размера базы (~$0,00005); retrieval recall@1 = 1,0 на всех четырёх размерах	Accuracy финального ответа колеблется без явного тренда (0,9 → 0,9 → 0,7 → 0,8) — нужный кусок находится исправно, но итоговый ответ не всегда повторяет этот идеальный recall
Класть в векторное хранилище	Эмбеддинг запроса сравнивается с эмбеддингами кусков базы, в промт идёт только найденный кусок	Промт держит ~257 токенов и ~$0,00005 независимо от размера базы; retrieval recall@1 = 1,0 на всех четырёх размерах, включая 500 документов	Требует отдельного шага подготовки (разбить базу, посчитать эмбеддинги) до того, как придёт первый вопрос; accuracy финального ответа тоже колеблется (0,9 → 0,8 → 0,9 → 0,7)

Где хранится база и что видит модель

Вместо того чтобы на каждый вопрос передавать модели всю базу целиком, процесс хранит базу отдельно от разговора. По каждому новому запросу он сначала находит в этом отдельном хранилище несколько подходящих кусков — и только их, не всю базу, — кладёт в промт.

Настоящий экран: векторное хранилище в n8n



Программно нарисованная схема в духе n8n — холст недостижим в этой среде ([#233-8]); имена узлов и значения — настоящие, со стенда.

И шаг эмбеддинга, и финальный вызов модели собраны обычным узлом HTTP Request с предустановленным типом учётных данных — отдельного штатного узла «эмбеддинг через OpenRouter» в каталоге n8n 2.42.7 нет.

Что видно Ольге, а что — обратившемуся

Обратившемуся это не видно вовсе — он получает тот же по качеству или лучший ответ. Ольге видно другое: при текущем объёме базы (пока он мал) разница почти не заметна, но именно от неё зависит, во сколько обходится каждый следующий ответ, когда база вырастет.

Принцип переносится на любой оркестратор, не только на n8n: промт, в который целиком кладётся база знаний, растёт в стоимости линейно с размером базы и не гарантирует роста точности — внешнее хранилище, из которого на каждый вопрос достаётся только нужный кусок, держит стоимость и объём промта постоянными независимо от того, сколько документов накопилось.

Когда хранилище окупается, а когда — нет

Критерий. Внешнее хранилище окупается, когда рост базы делает промт-стаффинг либо заметно дороже, либо структурно невозможным — а не самим фактом наличия базы.

КОГДА НУЖНО

- База растёт заметно быстрее, чем раз в год, и счёт идёт на сотни документов
- Стоимость вызова уже измеримо больше нуля и видна на счёте за месяц
- Размер базы может однажды превысить контекст модели

КОГДА НЕ НУЖНО

- База — те же десять-двадцать пунктов FAQ, что и на прошлом занятии, и расти не планирует
- Разница по стоимости ($0,0002 против $0,00005) ещё не оправдывает отдельный шаг подготовки
- До предела контекста далеко, и структурного отказа не предвидится

Устаревший индекс — типовая проблема ступени, не отдельная находка этого прогона. Векторное хранилище — это копия базы на момент, когда её туда положили, а не живая ссылка на источник. Если регламент в исходном документе поменялся, а переиндексация ещё не прошла, хранилище честно находит и отдаёт кусок — только устаревший.

Первая строка оси занятия закрыта: современная модель от роста базы не ломается структурно — она дорожает монотонно до определённого объёма, а затем перестаёт отвечать вовсе; внешнее хранилище держит стоимость и recall@1 = 1,0 постоянными на любом из проверенных размеров.

Кейс 2. Разбиение документа на части

Кейс 1 решил, где база будет жить, когда вырастет — в отдельном хранилище, не в промте. Но хранилище не глотает документ целиком: трёхстраничный регламент не укладывается в один кусок так, чтобы поиск потом мог точно указать на нужное место.

Как резать документ, чтобы кусок не терял смысл

Потребность. Документ разрезан так, что каждый кусок хранилища несёт законченный кусок смысла, и ответ на вопрос не оказывается разорван между двумя соседними кусками.

Версия 1 — естественная первая попытка, та, которую выбирают, если про это не думать отдельно: резать документ на части фиксированного размера — по счёту символов или токенов, — не оглядываясь на то, где в тексте проходят границы предложений или абзацев.

Самое простое правило реализации — один параметр, один проход по тексту. Сработает ли оно для смысла — проверяем дальше.

Где должен проходить разрез документа

Где должен проходить разрез документа?

резать строго по счёту символов, не глядя на текст · резать по границам предложений

Что показал прогон на пяти размерах куска

РАЗМЕР КУСКА, СИМВОЛОВ	V1 — РЕЗАТЬ ПО СЧЁТУ СИМВОЛОВ	V2 — РЕЗАТЬ ПО ПРЕДЛОЖЕНИЯМ
120	9,7% (7 из 72)	1,4% (1 из 72)
180 (заголовочное)	12,5% (9 из 72)	1,4% (1 из 72)
220	4,2% (3 из 72)	1,4% (1 из 72)
300	6,9% (5 из 72)	0,0% (0 из 72)
450	2,8% (2 из 72)	1,4% (1 из 72)

Версия 1 не монотонна по размеру куска: 9,7% → 12,5% → 4,2% → 6,9% → 2,8%. «Удобный» на вид размер не гарантирует качества без проверки.

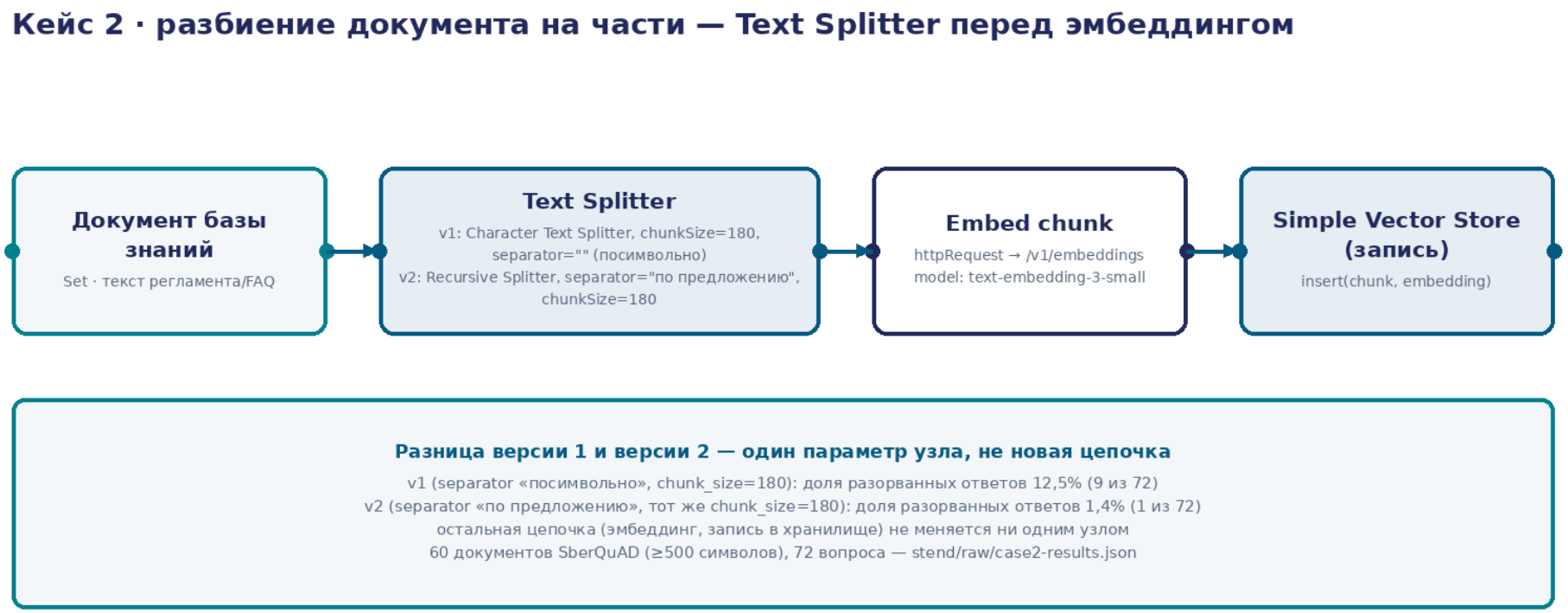
Что делает каждое из двух действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Резать по счёту символов	Документ рубится на части заданной длины без оглядки на текст внутри	Проще всего реализовать — один параметр, один проход	Разрывает ответ на вопрос в 2,8–12,5% случаев в зависимости от размера; неочевидно, какой размер «удобнее» без проверки
Резать по границам предложений	Разрез проходит только там, где заканчивается предложение	Доля разорванных ответов падает в разы — почти всегда 0–1,4% на всех проверенных размерах	Чуть сложнее в реализации (нужен разбор текста на предложения, не просто счёт символов); зависит от качества разбивки на предложения в конкретном языке и тексте

Где должна проходить граница куска

Граница куска должна совпадать с границей смысла внутри документа, а не с произвольным счётом символов. Резка, которая режет документ «как придётся», экономит один шаг подготовки и платит за это тем, что часть ответов на будущие вопросы окажется разорвана между двумя кусками.

Настоящий экран: узел Text Splitter



Программно нарисованная схема в духе n8n — холст недостижим в этой среде ([#233-8]); имена узлов и значения — настоящие, со стенда.

Остальная цепочка (шаг эмбеддинга из Кейса 1, запись в хранилище) не меняется ни одним узлом при переходе от версии 1 к версии 2.

Что видно Ольге, а что — обратившемуся

Обратившемуся результат виден напрямую: если ответ на его вопрос разорван между двумя кусками, модель получает только половину нужного факта и отвечает неполно или неверно — не потому, что плохо искала, а потому, что то, что она нашла, само по себе было неполным. Ольге видно число — какая доля вопросов рискует получить разорванный ответ при текущей настройке резки.

Принцип переносится на любой инструмент резки текста, не только на Text Splitter в n8n: кусок хранилища — это единица, внутри которой должен сохраняться смысл; выбор границы разреза — не техническая деталь конфигурации, а решение, которое прямо определяет, сколько ответов придут целыми.

Что этот замер не проверял

Размер куска не ведёт себя монотонно сам по себе. У версии 1 доля разорванных ответов не снижается плавно с ростом размера куска: 9,7% → 12,5% → 4,2% → 6,9% → 2,8%. Выбор «удобного на вид» размера не гарантирует качества — требуется проверка на своих данных, а не перенос чужого эмпирического правила.

Честный открытый пункт. Этот замер не варьировал перекрытие между соседними кусками — во всех прогонах оно было нулевым. Остаётся непроверенным, снижает ли добавленное перекрытие долю разорванных ответов версии 1 ещё дальше, или версия 2 делает его ненужным.

Вторая строка оси занятия закрыта: порезать документ на части МОЖНО как угодно технически — но от этого напрямую зависит, какая доля будущих ответов придёт целой.

Кейс 3. Чем искать нужный кусок

Кейс 2 разрезал документы правильно — по смыслу, не по счёту символов. Но правильно нарезанный кусок ещё нужно найти среди сотен таких же, когда приходит конкретный вопрос.

Как найти нужный кусок среди сотен таких же

Потребность. Среди сотен правильно нарезанных кусков базы найти ровно тот, в котором лежит ответ на заданный вопрос — и найти его на первом же месте выдачи, не на десятом.

Версия 1 — после того, что показали Кейсы 1 и 2 (поиск по смыслу работает, резка по смыслу работает), естественно предположить: искать тоже нужно по смыслу — через векторное сравнение запроса с кусками базы. Кажется, что раз такой поиск понимает смысл вопроса, он заведомо лучше поиска по совпадению слов.

Это и есть интуиция, которую кейс проверяет замером.

Чем искать нужный кусок среди сотен

Чем искать нужный кусок среди сотен?

искать по совпадению слов

искать по смыслу (вектор)

искать и тем, и тем, объединить результаты

отдать финальный порядок на пересмотр модели

Что показал прогон на 500 документах и 50 вопросах

МЕТОД	RECALL@1	RECALL@3	СРЕДНИЙ РАНГ ВЕРНОГО (ИЗ 500)
Искать по словам	0,74	0,80	9,9
Искать по смыслу (вектор)	0,86	0,92	2,0
Искать обоими, объединить (гибрид, RRF)	0,84	0,94	1,6

Пул из 500 документов содержит 8 «confusable clusters» — реальные SberQuAD-контексты, делящие редкую сущность (одна группа — про Боба Дилана, другая — про Сибирь). Вопросы отобраны кодом по трём классам: лексический разрыв, точное совпадение, нейтральные.

Что делает каждый из трёх сигналов

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Искать по словам	Совпадение токенов запроса и документа, без модели	Дёшево и быстро — поиск сам ничего не стоит	recall@1 = 0,74 — худший из трёх; если не на первом месте, в среднем на 9,9-й позиции из 500 — лексический разрыв и confusable-кластеры сбивают точное совпадение
Искать по смыслу (вектор)	Эмбеддинг запроса сравнивается с эмбеддингами кусков базы	recall@1 = 0,86 — лучший результат среди методов без модели на ранжировании; средний ранг 2,0	recall@3 (0,92) чуть ниже гибрида; слабее там, где важно точное совпадение имени или числа
Искать обоими, объединить (гибрид, RRF)	Два независимых ранжирования сливаются в одно по правилу Reciprocal Rank Fusion	Лучший recall@3 (0,94) и средний ранг (1,6) среди методов без модели на ранжировании	recall@1 = 0,84 — ниже, чем у чистого вектора (0,86): слабый сигнал слов иногда перетягивает объединённый голос не туда, где вектор был прав сразу

Следующий шаг после гибрида: пересмотр моделью

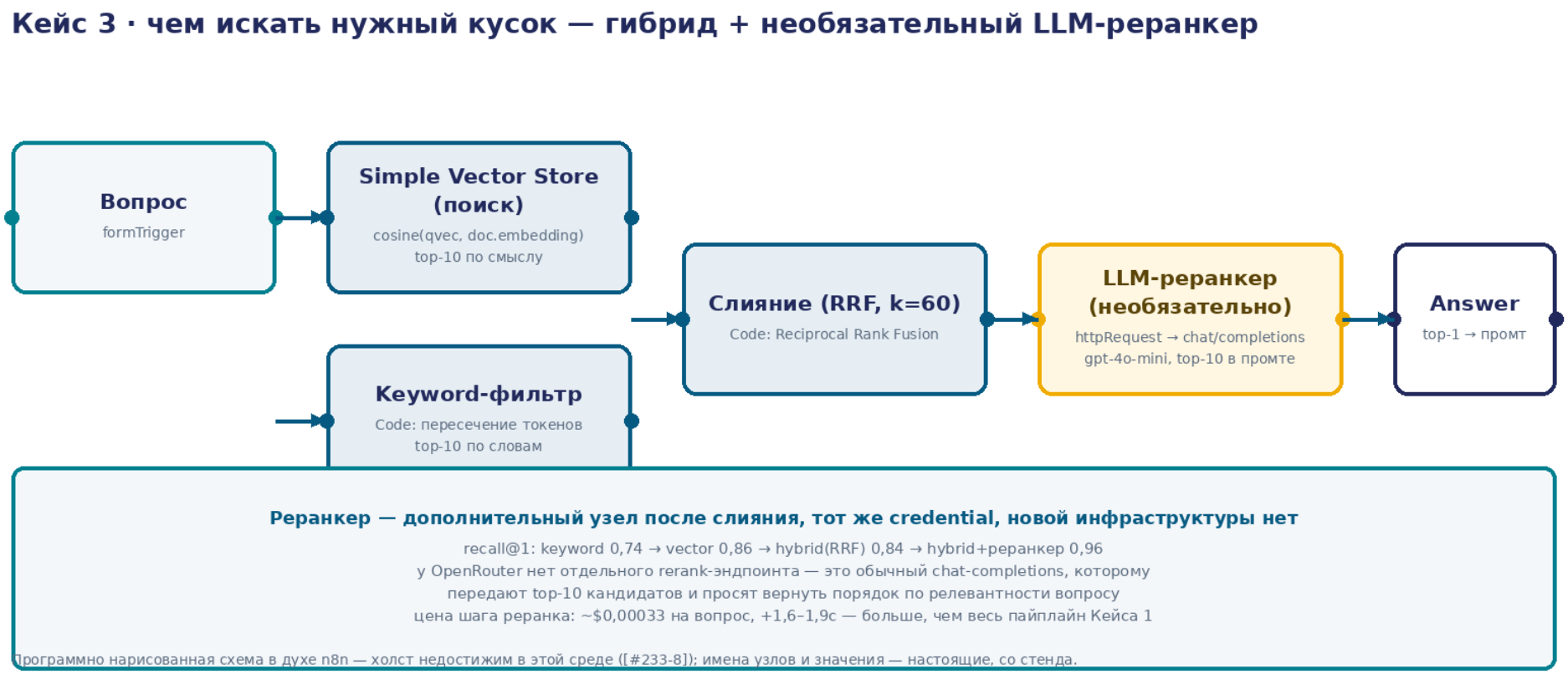
ОСНОВА + РЕРАНКЕР	RECALL@1	$/ВОПРОС (ПОИСК)
Гибрид + LLM-реранкер	**0,96** (было 0,84)	$0,000329
Вектор + LLM-реранкер	**0,96** (было 0,86)	$0,000327

Пример исправления. Вопрос «Какие наиболее популярные произведения были созданы на протяжении 1984–1986 гг» (gold-документ — член confusable-кластера): чистый вектор нашёл его на 10-м месте, гибрид — на 2-м; реранкер поднял оба варианта на 1-е.

Поиск — объединение сигналов, не один из них

Поиск — не один сигнал релевантности, а объединение минимум двух разных сигналов, прежде чем решать, какой кусок класть в промт. У одного сигнала (слова) своя область, где он выигрывает — точные совпадения, имена, числа; у другого (смысл) — своя — пересказ своими словами. После объединения сигналов — необязательный дополнительный шаг: отдать финальный порядок на пересмотр модели, если цена отдельного вызова и добавленная задержка оправданы ценой ошибки ранжирования.

Настоящий экран: слияние сигналов и реранкер



Вопрос идёт в векторное хранилище и в keyword-фильтр параллельно; два списка результатов сливаются одним узлом по правилу объединения рангов. Реранкер — отдельный шаг ПОСЛЕ слияния, не замена ему.

Что видно Ольге, а что — обратившемуся

Обратившемуся гибридный поиск виден как более стабильный ответ независимо от того, как именно сформулирован вопрос — точным термином или пересказом своими словами. Ольге видно число: доля вопросов, на которые процесс нашёл верный документ среди первых результатов, выше у гибрида, чем у любого из двух способов по отдельности.

Принцип переносится на любой оркестратор: поиск по смыслу не заменяет поиск по словам и не наоборот — у каждого свой класс вопросов, на которых он сильнее, и объединение обоих результатов не требует заранее знать, какой класс вопроса придёт следующим.

Когда гибрид не нужен и когда реранкер подводит

Критерий. Гибридный поиск — первый кандидат на снятие, если слот не вмещает весь материал: не потому что он не работает, а потому что на небольшой однородной базе он не нужен настолько, насколько нужен на большой и разнородной.

КОГДА НУЖНО

База большая, вопросы формулируются и точными терминами, и пересказом своими словами

Цена ошибки ранжирования высока — обратившийся получит неверный ответ по второму-третьему кандидату

КОГДА НЕ НУЖНО

База маленькая и однородная по формулировкам вопросов

Разница между одним способом и объединением двух статистически неразличима на этой базе

Реранкер — вторая граница того же рода, с двумя сторонами сразу. Он добавляет 10–12 процентных пунктов к recall@1 — но стоит отдельного вызова модели ($0,00033, дороже всего пайплайна Кейса 1) и задержки (+1,6–1,9с). Он может поднять gold-документ с 10-го места на 1-е (вопрос c3q10) — и столкнуть уже верный результат с 1-го на 3-е (вопрос c3q7, «Пример водных преград?»). Если документа нет даже среди top-10 кандидатов до реранка (вопрос c3q42, вектор нашёл gold на 26-м месте) — реранкер его не поднимет вовсе.

Третья строка оси занятия закрыта: поиск по смыслу находит документ на первом месте чаще, чем поиск по словам, на жёстком корпусе — но не гарантированно и не универсально; это зависит от корпуса и вопросов, а объединение сигналов не побеждает по каждой метрике сразу.

Кейс 4. Поиск по изображениям

Позапрошлое занятие показало пределы передачи картинки модели. Прошлое — что модель умеет её прочитать. Сегодня — та же задача хранения и поиска, что в трёх предыдущих кейсах, только для сканов и фотографий.

Хранение базы знаний | Разбиение документа на части | Чем искать нужный кусок | **Поиск по изображениям**

Как найти прошлый похожий скан

Потребность. Учебный центр получает сканы бланков и чеков в заявках регулярно. Нужно найти среди прошлых заявок похожий скан — не просто прочитать текущий.

Версия 1 — естественная первая попытка, прямое продолжение трёх предыдущих кейсов: подписать картинку текстом (распознать текст или получить текстовое описание от модели) и искать эту подпись тем же механизмом, что в Кейсе 3 — как обычный текстовый документ.

Один из трёх проверенных сегодня вариантов закончится честно показанным провалом — не ослабленной версией успеха, а настоящей неудачей с выученным уроком.

Как искать похожий скан среди прошлых

Как искать похожий скан среди прошлых?

подписать картинку словами и искать как текст

искать картинку по картинке напрямую

искать точные повторы отдельным быстрым способом

Что делает каждое из трёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Подписать и искать как текст	Текст чека превращается в тот же текстовый эмбеддинг, что в Кейсах 1-3	Работает корректно: эмбеддинги содержательно различаются между чеками, переиспользует весь механизм Кейсов 1-3	Подпись — пересказ картинки словами: расположение полей, визуальный вид бланка, печать, рукописная пометка теряются до того, как дошли до поиска
Искать картинку по картинке (мультимодальный эмбеддинг)	Модель превращает саму картинку в числовое представление без промежуточного текста	Теоретически не теряет ничего из того, что отбрасывает подпись	**На этом стенде не работает на реальном разрешении скана — разбор следующим тактом**
Искать точные повторы (perceptual hash)	Короткий числовой отпечаток картинки, вычисляемый локально, без модели	Надёжно ловит почти-дубли при пересжатии — без единого вызова модели	Решает только подзадачу «тот же самый скан, слегка пересжатый», не задачу «похожий по смыслу документ»

Провал, часть 1: модели плана не существует

Шаг 1. `voyage-multimodal-3`, модель, на которую план рассчитывал изначально, не найдена на OpenRouter этого стенда ни под одним из проверенных написаний id. Полный каталог моделей (452 позиции) не содержит ни одной модели с «voyage» или «embed» в названии вообще.

Перебор реальных кандидатов нашёл ровно одну рабочую мультимодальную модель — `amazon/titan-embed-image-v1` — она принимает и текст, и картинку, и отдаёт вектор без ошибки.

Провал, часть 2-3: ответ без ошибки ещё не значит «работает»

РАЗРЕШЕНИЕ КАРТИНКИ	ПОВЕДЕНИЕ
300×450 (реальный скан)	Битово ОДИНАКОВЫЙ вектор для любых двух разных картинок (cosine = 1,0 до 9 знаков)
128×192	Различие едва заметно (cosine ≈ 0,997 между разными картинками)
64×96 (маленькое превью)	Формально различает, но разброс между случайными чеками статистически неотличим от шума (корреляция −0,077)

Ответ без ошибки — не то же самое, что обработанный вход. Единственный способ это проверить — подать на вход две заведомо разные картинки и сравнить, различается ли выход, а не один тест «пришёл ли ответ правильной формы».

Правильная идея без готовой реализации

Правильная архитектура — сравнивать картинки в их собственном числовом представлении, не только через текстовый посредник. Это архитектурный тезис, и он остаётся верным независимо от того, какая конкретная модель его реализует.

Рабочее решение задачи на сегодняшнем стенде складывается из двух оставшихся частей разбора: подписать и искать как текст — для смыслового поиска «похожий документ по содержанию» — и perceptual hash отдельным быстрым шагом — для задачи «точно такой же скан, просто пересжат». Не компромисс вместо мультимодального эмбеддинга, а то же правило, что уже встретилось в Кейсе 3: один сигнал не покрывает все случаи.

Настоящий экран: попытка, которая не удалась



Программно нарисованная схема в духе n8n — холст недостижим в этой среде ([#233-8]); имена узлов и значения — настоящие, со стенда.

Экран на стенде реально собран и реально выполнен — но архитектурный тезис слоя 1 этим прогоном не подтверждён: рабочей демонстрации «поиск картинки по картинке» на этом стенде нет, только честно показанный и воспроизведённый провал.

Что видно Ольге, а что — обратившемуся

Обратившемуся не видно, как именно процесс нашёл прошлый похожий бланк — видно, что он его нашёл, когда это возможно по содержанию, и что система отдельно и надёжно отличает буквальный повторный скан той же заявки от нового документа. Ольге видно границу: для какого класса похожести (смысловая — «такой же тип бланка») нужен один механизм, а для какого (буквальная — «тот же самый документ, просто переснятый») — другой, и дешевле.

Принцип, переносимый на любой оркестратор: архитектурная идея «сравнивать данные в их собственном пространстве, не через пересказ» может быть верной и при этом не иметь готовой рабочей реализации на конкретном провайдере в конкретный момент — проверка этого занимает один контролируемый эксперимент с двумя разными входами, и пропустить этот эксперимент значит унести нерабочее решение, которое выглядело рабочим на первом тесте.

Где метод точных повторов перестаёт быть надёжным

ЧЕК	ДО «МЯГКОГО» ДУБЛЯ (БИТ ИЗ 64)	ДО «ЖЁСТКОГО» ДУБЛЯ	ДО СЛУЧАЙНОГО ЧУЖОГО (БАЗА)
receipt_07	1	27	14
receipt_18	0	10	24
receipt_17	0	30	35
receipt_04	1	40	23

Граница применимости — прямой формой. Под жёсткой трансформацией (поворот 12° + обрезка 4%) расстояние растёт заметно — до 10–40 бит, и в половине из шести проверенных чеков сопоставимо с расстоянием до случайного чужого чека. Метод решает подкласс «тот же скан, пересжатие или небольшой сдвиг» — и перестаёт быть надёжным ровно там, где меняется геометрия снимка, а не только его качество.

Четвёртая строка оси занятия закрыта: если умеем искать текст, картинку тем же способом не найдём — подпись теряет видимое, а прямое сравнение картинок может ответить без ошибки и при этом не использовать вход вовсе.

Релевантно ≠ верно

Четыре кейса сегодня учили находить нужный кусок базы знаний и класть его в промт перед ответом. Всё это время за кадром стоял не проверенный явно допуск: раз система нашла кусок, топически относящийся к вопросу, значит ответ на его основе окажется верным.

Случай 1. Google AI Overviews и совет про пиццу

В 2024 году функция обзоров с помощью ИИ в поиске Google несколько раз выдавала пользователям советы, взятые из источников, которые по форме звучали как достоверные, а по содержанию были шуткой: один из широко обсуждавшихся случаев — рекомендация добавлять нетоксичный клей в соус для пиццы, источником которой оказался старый шуточный комментарий на интернет-форуме.

Система нашла текст, который действительно был об этом — про сыр, про пиццу, про то, как сделать её менее текучей. По меркам поиска — успех: релевантность высокая. По меркам ответа пользователю — провал: источник был шуткой, а не кулинарной рекомендацией.

Случай 2. Чат-бот городской администрации Нью-Йорка

Отдельный, документированный журналистским расследованием случай 2024 года: городской чат-бот Нью-Йорка, созданный в помощь владельцам малого бизнеса, на прямые вопросы о городских правилах несколько раз выдавал уверенные, грамматически безупречные ответы, которые были юридически неверны или напрямую противоречили закону города — например, бот говорил, что можно забирать себе чаевые сотрудников, и что можно отказывать арендаторам с жилищными субсидиями.

Оба случая технически сработали правильно: нашли релевантный по теме текст и построили на его основе грамматически корректный ответ. Провал — не в механике поиска, а в допущении, что тематическая близость равна фактической верности.

Почему это касается всех четырёх кейсов сегодня

Ни Кейс 1 (хранение), ни Кейс 2 (резка), ни Кейс 3 (поиск), ни Кейс 4 (поиск по изображениям) не застрахованы от этого автоматически самим фактом, что они используют поиск вместо того, чтобы передавать модели весь объём данных целиком. Поиск, который правильно нашёл наиболее похожий по смыслу фрагмент — ровно то, что измеряли метрики recall и ранг во всех четырёх кейсах, — отвечает только на вопрос «это похоже на то, что спросили», не на вопрос «это верно».

Это не повод отказаться от поиска по базе знаний — отказ вернул бы занятие к промт-стаффингу, который Кейс 1 уже показал невыгодным при росте базы. Это повод для отдельного шага, который следующие занятия этой дуги строят отдельно: оценка качества и выключатель для ответа, построенного на найденном материале.

Четыре строки оси: чем оказалось

ВЫГЛЯДИТ НАДЁЖНЫМ	ЧЕМ ОКАЗЫВАЕТСЯ
если база выросла — просто перечислим её всю в промте, как на C9	Современная модель не ломается структурно сразу: accuracy падает монотонно (0,8→0,6→0,4), стоимость растёт ×16,7 — а на 500 документах запрос физически отклонён (HTTP 400, контекст превышен); поиск держит recall@1=1,0 и ~$0,00005 на всех размерах
порезать документ на части можно как угодно, главное чтобы кусок попал в хранилище	Резка по символам рвёт ответ в 2,8–12,5% случаев; резка по предложениям — почти всегда 0–1,4%; «удобный на вид» размер не гарантирует качества без проверки на своих данных
поиск по смыслу (вектор) и так находит то же самое, что поиск по словам, только умнее	На жёстком корпусе вектор обгоняет слова по recall@1 (0,86 vs 0,74) — но объединение сигналов не побеждает по каждой метрике разом; реранкер поднимает recall@1 до 0,96 ценой отдельного вызова модели и собственных ошибок
если умеем искать текст — картинку найдём тем же способом, просто подписав её словами	Подпись теряет то, что видно на картинке; прямое сравнение картинок на этом стенде технически не работает — модель отвечает без ошибки, но возвращает одинаковый вектор для разных изображений

Откуда процесс берёт факты и почему считает их верными?

Счёт занятия: что измерено, что осталось непроверенным

Четыре кейса, четыре самостоятельных замера на реальных данных — SberQuAD для текста (Кейсы 1-3), CORD для изображений (Кейс 4). Ни один — на придуманном числе. Один из четырёх прогонов — мультимодальный эмбеддинг в Кейсе 4 — закончился честно показанным нерабочим результатом, не доработанным суррогатом.

Хранилище (Кейс 1) экономит на стоимости ответа, но добавляет шаг подготовки данных и риск устаревшего индекса — копия базы не обновляется сама, когда меняется источник. Резка по смыслу (Кейс 2) точнее резки по счёту символов, но чуть сложнее в реализации и зависит от качества разбивки текста на предложения. Гибридный поиск (Кейс 3) надёжнее одного способа, но стоит лишнего шага вычисления и слияния на каждый запрос — и не оправдан на маленькой однородной базе; реранкер добавляет ещё один платный шаг сверху. Поиск по изображениям (Кейс 4) там, где он действительно нужен, требует отдельно проверенного компонента под каждый класс похожести — и ни один удобный универсальный ответ не оказался готов к использованию без собственной проверки.

Что дальше

Может ли процесс действовать от имени компании, не только отвечать?

Вопрос, с которого начался прошлый семинар («может ли процесс отвечать обратившемуся сам»), и вопрос, с которого начался сегодняшний («откуда процесс берёт факты»), вместе подводят к следующему шагу ступенчатой дуги курса: что происходит, когда процессу дают не только знание, но и полномочие — право не просто предложить ответ, а совершить действие от имени компании.

И то, чего сегодняшнее занятие не чинит: разбавитель этого дня — «найдено» ≠ «верно» — сам не закрывается ни выносом базы наружу, ни правом действовать. Это отдельная ступень: оценка качества и выключатель, дальше по дуге.